

The Temporal Radial Stack - a Domain-Agnostic Setup Guide

Status: working reference, distilled from the 2026-07 campaigns (animation / motion, character LM, word LM).
The one-sentence claim: a temporal radial stack learns sequence tasks with **zero gradients** - evolution searches feature *structure*, a closed-form ridge solves the *weights*, and time enters the model as ordinary channels, not as special machinery.

1. Why gradient-free, and where the learning actually lives

There is no backpropagation anywhere in this architecture. There isn't a hybrid trick either - no gradient sneaks in through a side door:

- **Feature genomes** are small programs (tens of numeric genes each).

They are found by evolutionary search: tournament selection, mutation, optional crossover, an energy economy. Nothing differentiates them.

- **The head** is a ridge regression solved in closed form (one linear

solve). It is the only "fitting" in the system, and it has an exact algebraic solution - no iterative optimization, no learning rate.

- **The environment** is built once from raw data statistics (PCA of

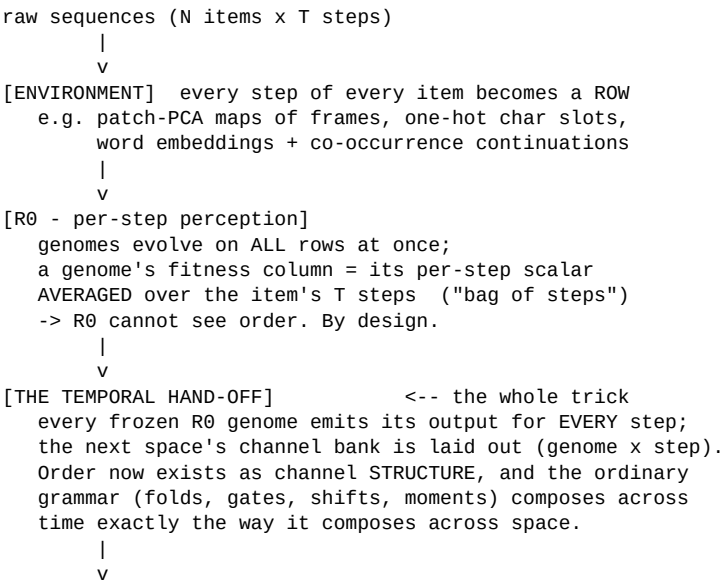
patches, PPMI of co-occurrence, SVD embeddings). It never sees labels.

The division of labor is the design's core idea:

Layer	Job	How it "learns"
Environment	everything tabulatable / statistical	computed once, from raw data only
Genomes	composition - relations the environment cannot tabulate	evolutionary search
Head	mapping features to classes	closed-form ridge (fp64 solve)

If a signal can be captured by a table or a linear map, do NOT ask evolution for it. Evolution is expensive; spend it only on what tables structurally cannot hold (interactions, relations, order).

2. The architecture in one picture



```

[R1..Rk - emergent-cap stacked spaces]
  standard spaces over the temporal bank; depth is decided
  by cap pressure, never by hand
    |
    v
[HEAD] closed-form ridge, C-way; test touched ONCE

```

The load-bearing insight: **time is not a new dimension to the grammar - it is just more channels.**

$\text{centroid}(\text{detector @ step5}) - \text{centroid}(\text{detector @ step0})$ is growth / motion / word order, and it is expressible with the same primitives that compose within a step.

3. Setup, step by step (any domain)

3.1 Shape the data

Produce $(N, T, \dots)$ sequences and a label per sequence. Reshape so that **every step is a row**: row index $n * T + t$. Keep a disjoint test region (different corpus region, held-out generator seeds - never a random split of near-duplicates).

3.2 Build the environment (labels forbidden here)

Pick the statistics that give step identity away for free:

- images/frames -> patch-PCA (Coates-Ng style) maps, several scales
- characters -> one-hot channels (use **dummy coding**: drop one

column per slot, or the slot-sum is collinear with the intercept and the solver dies at scale)

- words -> corpus-SVD embeddings + identity one-hots of the most

recent steps + **continuation channels** (the empirical next-step distribution from an INDEPENDENT corpus slice - fitting those tables on the training region leaks into validation and misleads evolution)

3.3 The anchor (your honesty instrument - TEST WITH IT, THEN REMOVE IT)

Fit the closed-form head on the raw environment channels with **no genomes at all**. This is the anchor. Use it for exactly one thing: **measurement**. It is the "zero of the ruler" - anything above it is what evolution earned - and during development you can put its columns in the border-ridge base to verify that a genome's gain is residual composition and not a re-derived table.

Then take it OUT of the production model - both out of the head and out of the fitness base. This was learned the hard way, twice over:

1. The anchor makes the head explode. Feeding thousands of raw

environment channels (identity one-hots, probability vectors) into a C-way head costs millions of parameters that do nothing but restate the environment linearly. Measured: a next-word model whose 4.8M parameters were 99.99% flat head and 66 evolved - a lookup table wearing a model's name.

2. The anchor makes the genomes lazy. With the anchor in the fitness

base, candidates must beat a strong linear model from their very first gene - nothing earns, spaces freeze one genome or none, and the stack never progresses past R0. Five configurations in a row produced flat genomes this way. Remove the anchor from the base (genomes earn freely, R0-style) and the same setup froze 334 genomes across 5 spaces, 37 of them attend genomes, at 14x fewer parameters.

The working pattern: **anchor as a reported baseline bar, never as a component**. The production head reads genome outputs (plus at most a small compressed environment summary if you must - never the raw identity/probability blocks).

Also compute the classical ceilings for your domain (n-grams for text, linear probes for vision) fit on the FULL training resource - give the baselines their best shot, or beating them means nothing.

3.4 R0 - per-step perception

Evolve genomes over single-step rows with fitness = the sequence-mean of the per-step scalar (the orderless bag). R0's base starts EMPTY - perception must earn freely; it should never fight the anchor. If the task's answer is visible within one step, R0 will solve it here and the stack will (correctly) refuse to go deeper.

3.5 The hand-off

For every frozen R0 genome, emit its per-step output and concatenate as $(N, n_genomes * T)$ channels (spatial tasks: keep the GRID - scalars strangle structure; measured: grid 0.870 vs scalar 0.850 and two extra productive spaces). **Standardize every frozen column by TRAIN statistics and clamp to ± 8 sd on BOTH sides** - a genome that is tame on train can explode on out-of-distribution data and one such column wrecks the head.

3.6 Deeper spaces under cap pressure

Each space evolves over `[environment skip bank | previous outputs]`. Do not set depth. Set a **pressure**: a space is FULL when a round's validation gain drops below the cap (a live-tunable file, e.g. 0.0002); the stack stops when a whole space earns less than `MIN_SPACE_GAIN`. The architecture then sizes itself to where the answer lives - measured cleanly on the twin experiment (same sequences, two labels): the motion label built 5 spaces with the temporal space largest; the shape label self-stopped at 2 spaces with R0 already perfect.

3.7 The head, once

Closed-form ridge over all frozen columns (+ anchor columns if you are running the fat variant). Pick lambda on the validation split. Touch the test set exactly once, at the end. Report top-1 AND top-k - sequence tasks are ambiguous and top-k is often the honest usefulness metric.

4. Numerical rails (each one bought with a real failure)

1. TF32 poisons grams. On Ampere+ GPUs fp32 matmul silently runs in

TF32 (~1e-3 relative error) - at $n=80k$ rows this swamps the ridge lambda and the Cholesky goes "not positive definite", or worse, the solve is quietly wrong and validation collapses with NO error. Disable TF32 (`torch.backends.cuda.matmul.allow_tf32 = False`).

2. Gram in true fp32, factor/solve in fp64. Pure-fp64 grams are

~60x slow on consumer GPUs (fp64 is rate-limited); true-fp32 gram + fp64 Cholesky/solve is fast AND stable. Retry the gram in fp64 only if the factorization ever fails.

3. Dummy-code one-hots. Slot-wise sum-to-one is exactly collinear with the intercept.

4. Sanitize every genome output (`nan_to_num + clamp`): exp-family primitives on gated values emit inf/NaN, and one poisoned column reads as a singular gram.

5. Never standardize with per-batch statistics. A single generation row has no batch stats (std of $n=1$ is NaN). Standardize with stored TRAIN statistics everywhere.

6. Check for zombie processes before diagnosing slowness. A killed wrapper can orphan its python child, which will silently share the GPU for hours. This has burned this lab twice.

5. Measured laws (respect them or re-measure them)

- **Self-sizing is real.** Depth follows where the answer lives (in-step vs between-steps) with identical configuration.

- **Scalar hand-offs destroy structure.** Pass grids/banks, not scalars.
- **Context dilution.** Widening the window dilutes uniform channel

search quadratically; wider context requires locality/drift mutation operators or content-based addressing, not just more slots.

- **The environment eats what is tabulatable.** If genomes look flat,

first ask whether the anchor already owns the signal - then give the genomes a space tables cannot reach (long-range relations, candidate-list reranking, cross-step conjunctions).

- **Cross-seed spread is +1.4 pts.** Single-seed deltas under ~1.5 pts

are soft evidence. Multi-seed or it didn't happen.

- **Seed unions are nearly-free accuracy** on perception tasks (the

CIFAR line went 0.7035 -> 0.7702 by unioning substrates), but verify transfer honestly - a union can also amplify region overfitting.

6. Reference results (all gradient-free, all test-touched-once)

Task	Wiring	Result	Baseline context
CIFAR-10 (full)	radial stack + unions	0.7702	raw-pixel ridge 0.324
Motion (10 paths, decoy shapes)	temporal hand-off	0.8971	chance 0.10
Shape (same clips, labels flipped)	self-stopped at 2 spaces	0.9989-1.0	chance 0.10; 5,344 params
Next-char (27-way)	positional channels	0.4060-0.4172	bigram 0.289 / trigram 0.415
Next-word V=2000, W=16	environment + head	0.3273 top-1 / 0.5843 top-5	trigram 0.182 / top-5 0.609
Next-word LEAN (genomes carry it)	attend + vec genomes	0.2201 @ 675K params	fat: 0.3276 @ ~9.6M params

7. Minimal skeleton (pseudocode)

```

rows = sequences.reshape(N * T, *step_shape)    # every step is a row
env   = build_environment(rows)                  # stats only, no labels
anchor = ridge(env.channels, labels)             # the honesty instrument

# R0: per-step perception, empty base, sequence-mean fitness
R0 = evolve(grammar, bank=env.channels_per_row,
            fitness=lambda g: ridge_gain(mean_over_T(g(rows))),
            base=[])

# THE HAND-OFF: order becomes channels
bank = concat([g(rows).reshape(N, T, ...) for g in R0], axis=channels)

# deeper spaces under cap pressure; base includes anchor -> residual-only
spaces = []
while True:
    S = evolve(grammar, bank=[env.skip | bank],
               base=[anchor | spaces], cap=read_live_cap())
    if gain(S) < MIN_SPACE_GAIN: break
    spaces.append(S); bank = concat([bank, S.outputs])

model = ridge([anchor | spaces], labels)          # closed form, fp64 solve
report(test_once(model), anchor, classical_ceilings, params_by_layer)

```

8. What to try when genomes come up flat

They will, on some substrates. In order:

1. Check whether the anchor is still in the fitness base or the head -

if it is, that IS the problem (see 3.3): remove it and let genomes earn freely.

2. Give evolution structure to address: content-based channel

addressing (attend genomes: an evolved query scores a key bank, softmax-attends over value channels - $W+2$ params, turns a needle-in-a-million conjunction into one gene).

3. Locality in mutation (channel drift), so structured banks are

searchable.

4. LEAN mode: remove the anchor from the head entirely and let genomes

carry the model - you lose absolute accuracy, you gain a model whose every parameter composes (measured: 14x fewer params at 67% of the fat accuracy, 4.6x better accuracy-per-parameter).